

Estructuras de Datos y Algoritmos

5 de junio de 2026

Tutoría 2

Autor: *Diego Banda* (diego.banda@mail.udp.cl)

1. Priority Queue

Para cierto algoritmo de aprendizaje supervisado (cositas que verán más adelante jeje) se requiere encontrar los k puntos más cercanos a un punto ingresado, se le pide a usted utilizar sus conocimientos para ejecutar tal búsqueda de puntos. Debe crear un código con el cual obtenga una lista de los puntos ordenados desde el más cercano hasta el más lejano.

- **Input:** N igual a los puntos en el espacio, k igual a los puntos a imprimir más cercanos, una lista de N puntos (con su identificador y con dos componentes cada uno) y después de estos N puntos, el nuevo al que se le buscará los cercanos.
- **Output:** Lista de los puntos más cercanos.
- **Ejemplo:**
 - **Input:**
 - 5 3
 - A 1 1
 - B 2 2
 - C 3 3
 - D 4 4
 - E 5 5
 - 6 6
 - **Output:** E D C

Se puede apoyar de la siguiente clase para cada punto:

```
1 public class Point {  
2     private char point;  
3     private int x;  
4     private int y;  
5  
6     public Point(char point, int x, int y) {  
7         this.point = point;  
8         this.x = x;  
9         this.y = y;  
10    }  
11 }
```

Listing 1: clase *point*

2. Set

Búsqueda de asistencia por el jefe, cálculo de big O y caso con dos listas.

Su jefe cada va al sistema de registros de llegada con la lista de los empleados a verificar si ya llegaron todos, sin embargo, se demora demasiado, ya que sigue el siguiente algoritmo: Mira en su lista el empleado y lo busca en la lista de empleados que ya llegaron, luego mira en siguiente en su lista y lo busca en la lista de empleados que ya llegaron. Responda:

1. Determine el Big O del algoritmo que sigue el jefe (suponiendo que la lista de empleados y lista de los que llegaron es de largo n).
2. Proponga una solución mejor utilizando set. Programela y justifique con Big O.

3. Map

```
1 public static int consultar(String consulta) {  
2     System.out.println("ejecutando consulta...");  
3     long tiempoInicio = System.currentTimeMillis();  
4     while (System.currentTimeMillis() - tiempoInicio < 10) {}  
5     return (int) (Math.random() * 101);  
6 }
```

Listing 2: Función *consultar*

Usted maneja el servidor de la empresa en la que trabaja, y para hacer el sistema mas eficiente, usted decide aplicar un caché a las consultas repetidas, sin embargo, al ser consultas aleatorias, no sabes cuales se realizarán. Implemente un sistema de caché utilizando map, a usted le llegarán consultas, en caso de no haberlas ejecutado antes, debe ejecutar la función *consultar(string consulta)* y guardar el output en su map, en caso de ya haber ejecutado esto antes, entregue la respuesta guardada.

- **Input:** Entero N con la cantidad de consultas, seguido de N palabras (consultas)

- **Output:** Resultado de cada consulta.

- **Ejemplo:**

- **Input:**

- 6
- hola
- chatgepete
- chatgepete
- google
- hola
- google

- **Output:**

- ejecutando consulta...
- 10
- ejecutando consulta...
- 57
- 57
- ejecutando consulta...
- 99
- 10

o 99

4. Heap

Programa un minHeap en Java utilizando ArrayList.

5. BST

5.1. preOrder, inOrder, postOrder

Ejecute la salida de *preOrder*, *inOrder*, y *postOrder* para el siguiente árbol:

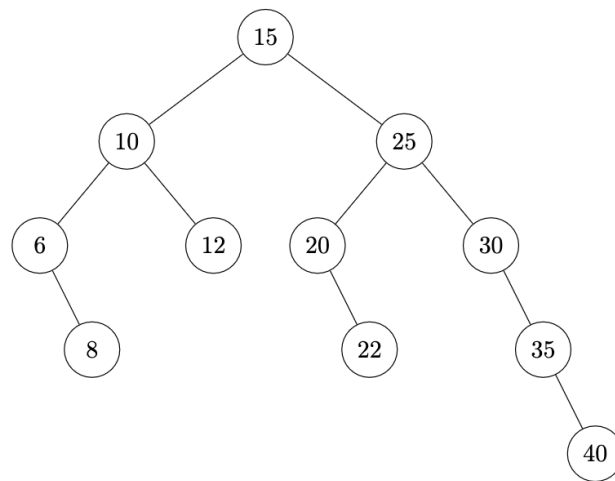


Figura 1: BST

5.2. Verificación

Usted es un ingeniero del área de QA, y como está muy aburrido pensando qué hacer durante el día, se le ocurre desarrollar un código en Java que valide la estructura de los BST implementados en la empresa.

- **Input:** Orden de entrada (puede ser “PreOrder”, “InOrder” o “PostOrder”).
- **Output:** Árbol correcto o no (True = correcto, False = incorrecto).
- **Ejemplo:**
 - **Input:** InOrder -78 -8 5 8 9 4
 - **Output:** False